Prática de Laboratório: Decisão vs. Busca e a Dificuldade do RSA

Disciplina: Análise de Algoritmos – Prof. Pedro Ximenes

Semestre 2026.2 – UNICAP

Instruções Gerais e Regra Obrigatória de Entrega

- **Momento da Aula:** Atividade de 15 a 20 minutos realizada em duplas imediatamente após os **Slides 22 a 25 da Aula 7** (Decisão vs. Busca, Fatoração e a Criptografia RSA).
- **Ambiente de Execução:** Terminal do laboratório ou compilador online: `https://www.online-python.com/`.
- **REGRA DE ENTREGA MANUSCRITA (OBRIGATÓRIO):**
  - **Todas as respostas e conclusões devem ser OBRIGATORIAMENTE redigidas À MÃO NO CADERNO (ou em folha de papel pautada para entrega física).**
  - **NÃO será aceita entrega digitada** (Word, Docs, PDF ou texto copiado).

---

1 Missão 1: A Assimetria do RSA (Multiplicação vs. Fatoração)

A segurança da internet baseia-se em uma assimetria: dados dois primos p e q, multiplicá-los para gerar $n = p \times q$ é trivial ($O(\log^2 n)$). Inversamente, dado apenas n, encontrar p e q é computacionalmente intratável por divisão por tentativa.

Instrução: Executem `codigo1_rsa_assimetria.py` e anotem os dados no caderno.

Tabela 1 – Medições Experimentais

Tamanho da Chave	Valor de $n = p \times q$	Tempo Mult.	Tempo Fatorar	Passos de Busca
16 bits ($p = 251, q = 257$)	64.507	_____	_____	_____
32 bits ($p = 65521, q = 65537$)	4.294.049.777	_____	_____	_____
42 bits ($p \approx 2 \times 10^6$)	4.398.063.288.167	_____	_____	_____
54 bits ($p \approx 6{,}7 \times 10^7$)	4.503.600.298.459.061	_____	_____	_____

Perguntas de Análise (Responder à mão no caderno)

1. Compare o tempo de multiplicação entre a chave de 16 bits e a de 54 bits. Por que o tempo de multiplicação quase não se altera (permanece em frações de microssegundo)?
2. Observe o tempo e o número de passos de fatoração conforme n cresce. O crescimento é polinomial ou exponencial em relação ao *número de bits* da entrada?
3. No RSA comercial de 2048 bits (n tem ~ 600 dígitos decimais), por que a divisão por tentativa é completamente inútil para quebrar a chave?

---

2 Missão 2: Decisão vs. Busca (Saber se é Composto vs. Achar os Fatores)

Nos **Slides 22 e 23**, vimos a diferença entre:

- **Decisão:** "O número n é composto?" → Resposta: **SIM** ou **NÃO** (Polinomial / Classe P).
- **Busca:** "Quais são os fatores primos de n?" → Encontrar p e q (Intratável / Busca Exponencial).

Instrução: Executem `codigo2_decisao_vs_busca.py` e anotem os tempos comparativos no caderno.

Tabela 2 – Decisão (Miller-Rabin) vs. Busca (Fatoração)

Instância	**Valor de n**	**Tempo Decisão (SIM/NÃO)**	**Tempo Busca (p, q)**
32 bits	4.294.049.777	_____	_____
42 bits	4.398.063.288.167	_____	_____
54 bits	4.503.600.298.459.061	_____	_____
70 bits	1.180.591.625.390.335.725.871	_____	_____

Perguntas de Análise (Responder à mão no caderno)

4. O teste de decisão conseguiu responder se o número de 70 bits é composto? Em quanto tempo? Ele forneceu algum dos fatores primos durante a execução?
5. Explique com suas próprias palavras o paradoxo: por que o computador pode provar que n é composto em milissegundos sem conseguir encontrar nenhum de seus divisores?

---

3 Missão 3: Encontrar vs. Verificar (Ponte para a Classe NP)

Instrução: Executem `codigo3_verificacao_certificado.py` e respondam no caderno:

6. Para $n = 1.180.591.625.390.335.725.871$, se alguém lhe entregar o certificado $p = 34.359.738.421$, quantas operações você precisa realizar para verificar se a afirmação é verdadeira? Quanto tempo isso levou?
7. Conclusão: por que problemas de busca difíceis podem ter verificação trivialmente rápida?